

Graph Algorithm Templates

`Queue<Integer> queue = new ArrayDeque<>()` throughout.
Six algorithms — each with a canonical template and representative problems.

Complexity Legend

BFS / DFS	$O(V + E)$	visit each vertex and edge once
Topo sort (Kahn)	$O(V + E)$	each node enqueued once, each edge relaxed once
Union-Find	$\sim O(n \cdot \alpha(n)) \approx O(n)$	α = inverse Ackermann, effectively constant
Dijkstra	$O((V + E) \log V)$	non-negative weights ONLY
Bellman-Ford	$O(V \cdot E)$	allows negative weights; run $k+1$ rounds for a k -hop limit
Prim's MST	$O(E \log V)$	PQ of candidate edges

Quick Reference Table

#	Name	Algorithm	Graph type	Key structure	Return	Time	Space
994	Rotting Oranges	Multi-source BFS	Grid	Queue<int[]>	int (minutes)	O(V+E)	O(V)
542	01 Matrix	Multi-source BFS	Grid	Queue<int[]> + dist[][]	int[][]	O(V+E)	O(V)
127	Word Ladder	BFS on state	Implicit	Queue<String> + Set	int (steps)	O(N·L·26)	O(N·L)
130	Surrounded Regions	BFS flood fill	Grid	Queue<int[]>	void	O(V+E)	O(V)
417	Pacific Atlantic	Multi-source BFS ×2	Grid	Queue<int[]> ×2	List<List<int>>>	O(V+E)	O(V)
207	Course Schedule	Topo sort (Kahn's)	Directed	int[] inDegree	boolean	O(V+E)	O(V+E)
210	Course Schedule II	Topo sort (Kahn's)	Directed	int[] inDegree	int[]	O(V+E)	O(V+E)
310	Minimum Height Trees	Topo sort (leaf trim)	Undirected	Set<Integer>[]	List<Integer>	O(V+E)	O(V+E)
547	Number of Provinces	Union Find	Undirected	int[] parent	int	O(n ² ·α(n))	O(n)
684	Redundant Connection	Union Find	Undirected	int[] parent	int[]	O(n·α(n))	O(n)
1319	Connect Network	Union Find	Undirected	int[] parent	int	O(E·α(n))	O(n)
743	Network Delay Time	Dijkstra	Directed weighted	PriorityQueue<int[]>	int	O((V+E)logV)	O(V+E)
787	Cheapest Flights K Stops	Bellman-Ford (k rounds)	Directed weighted	int[] dist per round	int	O(k·E)	O(V)
1514	Max Probability Path	Dijkstra (maximize)	Undirected weighted	PriorityQueue max-heap	double	O((V+E)logV)	O(V+E)
785	Is Graph Bipartite?	BFS 2-coloring	Undirected	int[] color	boolean	O(V+E)	O(V)
1584	Min Cost Connect Points	Prim's MST	Complete graph	PriorityQueue<int[]>	int	O(n ² logn)	O(n ²)
261	Graph Valid Tree	Check if n nodes and edges form a valid tree	Union Find: valid tree has exactly n-1 edges AND no cycles	Tree check: edges.length == n-1 is necessary; UF detects cycles	boolean	O(n·α(n))	O(n)
269	Alien Dictionary	Determine character order from sorted alien words	Build directed graph from adjacent word pairs; topological sort	Topo sort on char graph: compare adjacent words letter-by-letter to extract ordering edges	String	O(C) C=total chars	O(1)
286	Walls and Gates	Fill each empty room with distance to nearest gate	Multi-source BFS from all gates simultaneously	Multi-source BFS: seed queue with ALL gates at distance 0	void	O(m·n)	O(m·n)
323	Number of Connected Components	Count connected components in undirected graph	Union Find: union all edges; count distinct roots	Standard UF component count: start with n, decrement per successful union	int	O(n·α(n))	O(n)
721	Accounts Merge	Merge accounts sharing any common email	Union Find on emails; group by root	Union Find on strings: map each email to an id, union emails within an account	List<List<String>>>	O(n·α)	O(n)
827	Making A Large Island	Flip one 0 to 1 to maximize island size	Label islands with id+size map; for each 0 sum unique neighbor island sizes +1	Two-pass labeling: first DFS-color islands, then test each 0	int	O(m·n)	O(m·n)
909	Snakes and Ladders	Min moves to reach last square (board with snakes/ladders)	BFS on flattened board; convert square number to (row,col) via boustrophedon	BFS on board cells: number→coordinate conversion for zigzag rows	int	O(n ²)	O(n ²)
1091	Shortest Path in Binary Matrix	Shortest 8-directional path top-left to bottom-right through 0s	BFS with 8 directions	8-directional BFS: include diagonals in dr/dc	int	O(m·n)	O(m·n)
1631	Path With Minimum Effort	Min effort path from top-left to bottom-right; effort = max abs difference along path	Dijkstra with effort[r][c] = min-so-far max diff; PQ sorted by effort	Modified Dijkstra: dist = max(currEffort, edgeCost) instead of sum	int	O(m·n·log(m·n))	O(m·n)

Graph Representation

```
// Adjacency list (most common)
List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
for (int[] edge : edges) {
    graph.get(edge[0]).add(edge[1]);
    graph.get(edge[1]).add(edge[0]);    // omit for directed
}

// Weighted adjacency list: int[] = {neighbor, weight}
List<List<int[]>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
for (int[] edge : edges)
    graph.get(edge[0]).add(new int[]{edge[1], edge[2]});

// Grid 4-directional neighbors
int[] dr = {1, -1, 0, 0};
int[] dc = {0, 0, 1, -1};
```

All Six Templates

```
// 1. BFS – shortest path / level order
// MENTAL MODEL: explore in rings of equal distance, so the first time you reach a node is the shortest.
// WHEN: "fewest steps", "shortest path" on an unweighted graph
Queue<Integer> queue = new ArrayDeque<>();
boolean[] visited = new boolean[n];
queue.offer(start);
visited[start] = true;
while (!queue.isEmpty()) {
    int node = queue.poll();
    for (int neighbor : graph.get(node)) {
        if (!visited[neighbor]) {
            visited[neighbor] = true;
            queue.offer(neighbor);
        }
    }
}

// 2. TOPOLOGICAL SORT – Kahn's BFS
// MENTAL MODEL: peel off nodes with no remaining prerequisites; if any get stuck, a cycle exists.
// WHEN: "valid order", "dependencies/prerequisites", "detect cycle" on a directed graph
int[] inDegree = new int[n];
for (int[] edge : edges) inDegree[edge[1]]++;
Queue<Integer> queue = new ArrayDeque<>();
for (int i = 0; i < n; i++) if (inDegree[i] == 0) queue.offer(i);
List<Integer> order = new ArrayList<>();
while (!queue.isEmpty()) {
    int node = queue.poll();
    order.add(node);
    for (int neighbor : graph.get(node))
        if (--inDegree[neighbor] == 0) queue.offer(neighbor);
}
// order.size() == n → no cycle

// 3. UNION FIND
// MENTAL MODEL: each set points to one representative; merge sets by linking roots, query by finding roots.
// WHEN: "connected components", "is it already connected", "merge groups dynamically"
int[] parent, rank;
void init(int n) {
    parent = new int[n]; rank = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
}
int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]); // path compression
    return parent[x];
}
boolean union(int x, int y) {
    int px = find(x), py = find(y);
    if (px == py) return false;
    if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
    parent[py] = px;
    if (rank[px] == rank[py]) rank[px]++;
    return true;
}

// 4. DIJKSTRA – shortest path (non-negative weights)
// MENTAL MODEL: greedily settle the closest unfinished node; non-negative weights guarantee it's final.
// WHEN: "shortest/cheapest path" with non-negative weights
int[] dist = new int[n];
Arrays.fill(dist, Integer.MAX_VALUE);
dist[src] = 0;
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); // min-heap on cost
pq.offer(new int[]{src, 0});
while (!pq.isEmpty()) {
```

```

    int[] current = pq.poll();
    int node = current[0], d = current[1];
    if (d > dist[node]) continue;    // stale entry
    for (int[] nb : graph.get(node)) {
        int next = nb[0], w = nb[1];
        if (dist[node] + w < dist[next]) {
            dist[next] = dist[node] + w;
            pq.offer(new int[]{next, dist[next]});
        }
    }
}

// 5. BIPARTITE CHECK – BFS 2-coloring
// MENTAL MODEL: paint neighbors the opposite color; a neighbor that's already your color means an odd cycle.
// WHEN: "split into two groups", "2-colorable", "no edge within a group"
int[] color = new int[n];
Arrays.fill(color, -1);
Queue<Integer> queue = new ArrayDeque<>();
for (int start = 0; start < n; start++) {
    if (color[start] != -1) continue;
    color[start] = 0;
    queue.offer(start);
    while (!queue.isEmpty()) {
        int node = queue.poll();
        for (int neighbor : graph.get(node)) {
            if (color[neighbor] == -1) { color[neighbor] = 1 - color[node]; queue.offer(neighbor); }
            else if (color[neighbor] == color[node]) return false; // conflict
        }
    }
}
return true;

// 6. PRIM'S MST – greedy, always add cheapest edge to growing tree
// MENTAL MODEL: grow one tree outward, each step absorbing the cheapest edge that reaches a new node.
// WHEN: "connect all nodes at minimum total cost"
boolean[] inMST = new boolean[n];
int[] minEdge = new int[n];
Arrays.fill(minEdge, Integer.MAX_VALUE);
minEdge[0] = 0;
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
pq.offer(new int[]{0, 0});
int total = 0;
while (!pq.isEmpty()) {
    int[] current = pq.poll();
    int node = current[0], cost = current[1];
    if (inMST[node]) continue;
    inMST[node] = true;
    total += cost;
    for (int[] nb : graph.get(node))
        if (!inMST[nb[0]] && nb[1] < minEdge[nb[0]]) {
            minEdge[nb[0]] = nb[1];
            pq.offer(new int[]{nb[0], nb[1]});
        }
}
}

```

Part 1 — BFS Shortest Path

Single-Source vs Multi-Source

Single-source: one starting node, find distances to all others

Multi-source: seed queue with ALL starting nodes at once → BFS fans out from all simultaneously
→ avoids nested loops, handles "nearest X" questions in $O(n)$

#994 Rotting Oranges

Description: Grid of 0 (empty), 1 (fresh), 2 (rotten). Each minute rotten oranges infect adjacent fresh ones. Return minutes until no fresh remain, or -1.

Algorithm: Multi-source BFS — seed queue with all rotten cells simultaneously.

```
class Solution {
    public int orangesRotting(int[][] grid) {
        int rows = grid.length, cols = grid[0].length;
        Queue<int[]> queue = new ArrayDeque<>();
        int fresh = 0;
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++) {
                if (grid[r][c] == 2) queue.offer(new int[]{r, c});
                if (grid[r][c] == 1) fresh++;
            }
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        int minutes = 0;
        while (!queue.isEmpty() && fresh > 0) {
            minutes++;
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                int[] current = queue.poll();
                for (int d = 0; d < 4; d++) {
                    int nr = current[0]+dr[d], nc = current[1]+dc[d];
                    if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && grid[nr][nc] == 1) {
                        grid[nr][nc] = 2;
                        queue.offer(new int[]{nr, nc});
                        fresh--;
                    }
                }
            }
        }
        return fresh == 0 ? minutes : -1;
    }
}
```

Time $O(V+E)$ | **Space** $O(V)$

#542 01 Matrix

Description: For each cell, return the distance to the nearest 0.

Algorithm: Multi-source BFS from all 0 cells simultaneously. Initialize $dist[r][c] = MAX$ for 1 cells.

```

class Solution {
    public int[][] updateMatrix(int[][] mat) {
        int rows = mat.length, cols = mat[0].length;
        int[][] dist = new int[rows][cols];
        Queue<int[]> queue = new ArrayDeque<>();
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++) {
                if (mat[r][c] == 0) {
                    queue.offer(new int[]{r, c});
                } else {
                    dist[r][c] = Integer.MAX_VALUE;
                }
            }
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            for (int d = 0; d < 4; d++) {
                int nr = current[0]+dr[d], nc = current[1]+dc[d];
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols
                    && dist[current[0]][current[1]] + 1 < dist[nr][nc]) {
                    dist[nr][nc] = dist[current[0]][current[1]] + 1;
                    queue.offer(new int[]{nr, nc});
                }
            }
        }
        return dist;
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#127 Word Ladder

Description: Minimum number of transformations from `beginWord` to `endWord` , changing one letter at a time. Each intermediate word must be in `wordList` .

Algorithm: BFS on word states. Remove words from set when visited to prevent re-visiting.

```

class Solution {
    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        Set<String> wordSet = new HashSet<>(wordList);
        if (!wordSet.contains(endWord)) return 0;
        Queue<String> queue = new ArrayDeque<>();
        queue.offer(beginWord);
        wordSet.remove(beginWord);
        int steps = 1;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                char[] word = queue.poll().toCharArray();
                for (int j = 0; j < word.length; j++) {
                    char original = word[j];
                    for (char c = 'a'; c <= 'z'; c++) {
                        word[j] = c;
                        String next = new String(word);
                        if (next.equals(endWord)) return steps + 1;
                        if (wordSet.contains(next)) {
                            wordSet.remove(next);
                            queue.offer(next);
                        }
                    }
                    word[j] = original;
                }
            }
            steps++;
        }
        return 0;
    }
}

```

Time $O(N \cdot L \cdot 26)$ where N = words, L = word length | **Space** $O(N \cdot L)$

#130 Surrounded Regions

Description: Flip all '0' regions not connected to the border to 'X'.

Algorithm: Multi-source BFS from all border '0' cells. Mark reachable as safe ('S'), then flip all remaining '0' to 'X' and restore 'S' to '0'.


```

class Solution {
    public void solve(char[][] board) {
        int rows = board.length, cols = board[0].length;
        Queue<int[]> queue = new ArrayDeque<>();
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++)
                if ((r == 0 || r == rows-1 || c == 0 || c == cols-1) && board[r][c] == '0') {
                    board[r][c] = 'S';
                    queue.offer(new int[]{r, c});
                }
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            for (int d = 0; d < 4; d++) {
                int nr = current[0]+dr[d], nc = current[1]+dc[d];
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && board[nr][nc] == '0') {
                    board[nr][nc] = 'S';
                    queue.offer(new int[]{nr, nc});
                }
            }
        }
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++)
                board[r][c] = board[r][c] == 'S' ? '0' : 'X';
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#417 Pacific Atlantic Water Flow

Description: Return all cells from which water can flow to both the Pacific (top/left border) and Atlantic (bottom/right border).

Algorithm: Two separate multi-source BFS — one from each ocean's border cells, moving **uphill** (water flows down, so reachable cells going up = cells that drain down to ocean). Answer = intersection of both reachable sets.

```

class Solution {
    int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};

    public List<List<Integer>> pacificAtlantic(int[][] h) {
        int rows = h.length, cols = h[0].length;
        boolean[][] pac = new boolean[rows][cols];
        boolean[][] atl = new boolean[rows][cols];
        Queue<int[]> pacQ = new ArrayDeque<>(), atlQ = new ArrayDeque<>();
        for (int r = 0; r < rows; r++) {
            pac[r][0] = atl[r][cols-1] = true;
            pacQ.offer(new int[]{r, 0});
            atlQ.offer(new int[]{r, cols-1});
        }
        for (int c = 0; c < cols; c++) {
            pac[0][c] = atl[rows-1][c] = true;
            pacQ.offer(new int[]{0, c});
            atlQ.offer(new int[]{rows-1, c});
        }
        bfs(h, pacQ, pac);
        bfs(h, atlQ, atl);
        List<List<Integer>> result = new ArrayList<>();
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++)
                if (pac[r][c] && atl[r][c]) result.add(Arrays.asList(r, c));
        return result;
    }

    private void bfs(int[][] h, Queue<int[]> queue, boolean[][] visited) {
        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            for (int d = 0; d < 4; d++) {
                int nr = current[0]+dr[d], nc = current[1]+dc[d];
                if (nr >= 0 && nr < h.length && nc >= 0 && nc < h[0].length
                    && !visited[nr][nc] && h[nr][nc] >= h[current[0]][current[1]]) {
                    visited[nr][nc] = true;
                    queue.offer(new int[]{nr, nc});
                }
            }
        }
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

Part 2 — Topological Sort (Kahn's BFS)

Core idea: repeatedly remove nodes with `inDegree == 0` (no prerequisites). If all nodes removed $\rightarrow$ no cycle. Post-order in DFS = reverse of `order` here.

#207 Course Schedule

Description: Can you finish all courses given prerequisites? Detect if there is a directed cycle.

```

class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        List<List<Integer>> graph = new ArrayList<>();
        int[] inDegree = new int[numCourses];
        for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
        for (int[] p : prerequisites) {
            graph.get(p[1]).add(p[0]);
            inDegree[p[0]]++;
        }
        Queue<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < numCourses; i++) if (inDegree[i] == 0) queue.offer(i);
        int processed = 0;
        while (!queue.isEmpty()) {
            int node = queue.poll();
            processed++;
            for (int neighbor : graph.get(node))
                if (--inDegree[neighbor] == 0) queue.offer(neighbor);
        }
        return processed == numCourses;    // all processed → no cycle
    }
}

```

Time $O(V+E)$ | **Space** $O(V+E)$

#210 Course Schedule II

Description: Return a valid order to take all courses. Return `[]` if impossible.

```

class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        List<List<Integer>> graph = new ArrayList<>();
        int[] inDegree = new int[numCourses];
        for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
        for (int[] p : prerequisites) {
            graph.get(p[1]).add(p[0]);
            inDegree[p[0]]++;
        }
        Queue<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < numCourses; i++) if (inDegree[i] == 0) queue.offer(i);
        int[] order = new int[numCourses];
        int idx = 0;
        while (!queue.isEmpty()) {
            int node = queue.poll();
            order[idx++] = node;
            for (int neighbor : graph.get(node))
                if (--inDegree[neighbor] == 0) queue.offer(neighbor);
        }
        return idx == numCourses ? order : new int[]{};
    }
}

```

Time $O(V+E)$ | **Space** $O(V+E)$

#310 Minimum Height Trees

Description: Find all roots that minimize tree height. At most 2 answers — they are the center node(s) of the longest path.

Algorithm: Iteratively trim leaves (degree-1 nodes) until ≤ 2 nodes remain — same as topo sort but on undirected tree.

Intuition: the best root sits at the center of the longest path; peeling leaves layer by layer converges there.

```

class Solution {
    public List<Integer> findMinHeightTrees(int n, int[][] edges) {
        if (n == 1) return Collections.singletonList(0);
        List<Set<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) graph.add(new HashSet<>());
        for (int[] e : edges) { graph.get(e[0]).add(e[1]); graph.get(e[1]).add(e[0]); }
        Queue<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < n; i++) if (graph.get(i).size() == 1) queue.offer(i);
        int remaining = n;
        while (remaining > 2) {
            int size = queue.size();
            remaining -= size;
            for (int i = 0; i < size; i++) {
                int leaf = queue.poll();
                int neighbor = graph.get(leaf).iterator().next();
                graph.get(neighbor).remove(leaf);
                if (graph.get(neighbor).size() == 1) queue.offer(neighbor);
            }
        }
        return new ArrayList<>(queue);
    }
}

```

Time $O(V+E)$ | **Space** $O(V+E)$

207 vs 210 — Side by Side

	#207 (boolean)	#210 (order)
Track:	int processed	int[] order, int idx
Return:	processed == n	idx == n ? order : []
Difference:	just count	also record node into order[]

Part 3 — Union Find

Template (always the same three methods):

```

int[] parent, rank;

void init(int n) {
    parent = new int[n];
    rank = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
}

int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]); // path compression
    return parent[x];
}

boolean union(int x, int y) { // returns false if already connected
    int px = find(x), py = find(y);
    if (px == py) return false;
    if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
    parent[py] = px;
    if (rank[px] == rank[py]) rank[px]++;
    return true;
}

```

#547 Number of Provinces

Description: Count the number of connected components (provinces) in an undirected graph given as an adjacency matrix.

Intuition: start with n separate sets; every successful merge drops the count by one.

```
class Solution {
    int[] parent, rank;
    public int findCircleNum(int[][] isConnected) {
        int n = isConnected.length;
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
        int components = n;
        for (int i = 0; i < n; i++)
            for (int j = i+1; j < n; j++)
                if (isConnected[i][j] == 1 && union(i, j)) components--;
        return components;
    }
    int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }
    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }
}
```

Time $O(n^2 \cdot \alpha(n))$ | **Space** $O(n)$

#684 Redundant Connection

Description: Find the edge that creates a cycle in an undirected graph that would otherwise be a tree.

Key: try to union each edge in order. The first edge where both nodes already share a root is the redundant one.

Intuition: if both endpoints are already in the same set, this edge closes a cycle — it's the redundant one.

```
class Solution {
    int[] parent, rank;
    public int[] findRedundantConnection(int[][] edges) {
        int n = edges.length;
        parent = new int[n+1]; rank = new int[n+1];
        for (int i = 0; i <= n; i++) parent[i] = i;
        for (int[] edge : edges)
            if (!union(edge[0], edge[1])) return edge; // already connected → cycle
        return new int[]{};
    }
    int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }
    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }
}
```

Time $O(n \cdot \alpha(n))$ | **Space** $O(n)$

#1319 Number of Operations to Make Network Connected

Description: Minimum cable moves to connect all n computers. Return -1 if impossible.

Key: need at least $n-1$ edges for n nodes. Count extra edges (cycles); count components. Answer = $\text{components} - 1$ if enough extras exist.

Intuition: each redundant cable can be moved to bridge one gap, so `components - 1` moves suffice if you have enough spare cables.

```
class Solution {
    int[] parent, rank;
    public int makeConnected(int n, int[][] connections) {
        if (connections.length < n - 1) return -1;    // not enough cables
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
        int components = n;
        for (int[] c : connections) if (union(c[0], c[1])) components--;
        return components - 1;
    }
    int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }
    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }
}
```

Time $O(E \cdot \alpha(n))$ | **Space** $O(n)$

Part 4 — Dijkstra / Bellman-Ford

#743 Network Delay Time

Description: Time for a signal from `k` to reach all `n` nodes. Return -1 if unreachable.

Algorithm: Dijkstra from source `k`. Answer = max of all shortest distances.

Intuition: every node is reached by its shortest delay; the network is "done" when the slowest of those arrives.

```
class Solution {
    public int networkDelayTime(int[][] times, int n, int k) {
        List<List<int[]>> graph = new ArrayList<>();
        for (int i = 0; i <= n; i++) graph.add(new ArrayList<>());
        for (int[] t : times) graph.get(t[0]).add(new int[]{t[1], t[2]});
        int[] dist = new int[n+1];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[k] = 0;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
        pq.offer(new int[]{k, 0});
        while (!pq.isEmpty()) {
            int[] current = pq.poll();
            int node = current[0], d = current[1];
            if (d > dist[node]) continue;
            for (int[] nb : graph.get(node)) {
                int next = nb[0], w = nb[1];
                if (dist[node] + w < dist[next]) {
                    dist[next] = dist[node] + w;
                    pq.offer(new int[]{next, dist[next]});
                }
            }
        }
        int maxDist = 0;
        for (int i = 1; i <= n; i++) {
            if (dist[i] == Integer.MAX_VALUE) return -1;
            maxDist = Math.max(maxDist, dist[i]);
        }
        return maxDist;
    }
}
```

Time $O((V+E)\log V)$ | **Space** $O(V+E)$

#787 Cheapest Flights Within K Stops

Description: Cheapest price from `src` to `dst` using at most `k` stops (`k+1` edges).

Algorithm: Bellman-Ford with exactly `k+1` relaxation rounds. Use a copy `temp[]` each round to prevent using the same edge twice in one round.

Intuition: each round lets paths grow by one more edge, so `k+1` rounds caps the path at `k` stops.

```
class Solution {
    public int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
        int[] dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;
        for (int round = 0; round <= k; round++) {           // k stops = k+1 edges
            int[] temp = Arrays.copyOf(dist, n);
            for (int[] f : flights) {
                int u = f[0], v = f[1], w = f[2];
                if (dist[u] != Integer.MAX_VALUE && dist[u] + w < temp[v])
                    temp[v] = dist[u] + w;
            }
            dist = temp;
        }
        return dist[dst] == Integer.MAX_VALUE ? -1 : dist[dst];
    }
}
```

Time $O(k \cdot E)$ | **Space** $O(V)$

#1514 Path with Maximum Probability

Description: Find the path from `start` to `end` with maximum success probability (product of edge probabilities).

Algorithm: Dijkstra with **max-heap** (maximize probability instead of minimize distance). Multiply probabilities instead of adding weights.

Intuition: probabilities in $[0, 1]$ only shrink when multiplied, so the "closest" (highest-probability) node settles first — Dijkstra still applies.

```

class Solution {
    public double maxProbability(int n, int[][] edges, double[] succProb, int start, int end) {
        List<List<double[]>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
        for (int i = 0; i < edges.length; i++) {
            int u = edges[i][0], v = edges[i][1];
            graph.get(u).add(new double[]{v, succProb[i]});
            graph.get(v).add(new double[]{u, succProb[i]});
        }
        double[] prob = new double[n];
        prob[start] = 1.0;
        PriorityQueue<double[]> pq = new PriorityQueue<>((a, b) -> Double.compare(b[1], a[1])); // max-heap
        pq.offer(new double[]{start, 1.0});
        while (!pq.isEmpty()) {
            double[] current = pq.poll();
            int node = (int) current[0];
            double p = current[1];
            if (node == end) return p;
            if (p < prob[node]) continue;
            for (double[] nb : graph.get(node)) {
                int next = (int) nb[0];
                double newProb = p * nb[1];
                if (newProb > prob[next]) {
                    prob[next] = newProb;
                    pq.offer(new double[]{next, newProb});
                }
            }
        }
        return 0.0;
    }
}

```

Time $O((V+E)\log V)$ | **Space** $O(V+E)$

Dijkstra vs Bellman-Ford

	Dijkstra	Bellman-Ford
Weights:	Non-negative only	Any (handles negative)
Constraint:	—	k-hop limit (run k+1 rounds)
Time:	$O((V+E) \log V)$	$O(k \cdot E)$
When to use:	Standard shortest path	k-stop limit or negative weights
Key trick:	stale check: $d > \text{dist}[\text{node}]$	copy array each round (temp[])

Part 5 — Bipartite Check

#785 Is Graph Bipartite?

Description: Can graph nodes be split into two groups where every edge connects nodes in different groups?

Algorithm: BFS 2-coloring. If any neighbor has the same color as the current node → not bipartite.

Key: run BFS from every unvisited node (graph may be disconnected).

Intuition: paint each neighbor the opposite color; a clash means an odd-length cycle, which can't be 2-colored.


```

class Solution {
    public boolean isBipartite(int[][] graph) {
        int n = graph.length;
        int[] color = new int[n];
        Arrays.fill(color, -1);
        Queue<Integer> queue = new ArrayDeque<>();
        for (int start = 0; start < n; start++) {
            if (color[start] != -1) continue;
            color[start] = 0;
            queue.offer(start);
            while (!queue.isEmpty()) {
                int node = queue.poll();
                for (int neighbor : graph[node]) {
                    if (color[neighbor] == -1) {
                        color[neighbor] = 1 - color[node];
                        queue.offer(neighbor);
                    } else if (color[neighbor] == color[node]) {
                        return false;
                    }
                }
            }
        }
        return true;
    }
}

```

Time $O(V+E)$ | **Space** $O(V)$

Part 6 — Minimum Spanning Tree (Prim's)

#1584 Min Cost to Connect All Points

Description: Connect all points (on a 2D plane) with minimum total Manhattan distance. Return that total cost.

Algorithm: Prim's MST — always add the cheapest edge from the growing tree to a new node.

Key: `minEdge[i]` tracks the cheapest known edge from any tree node to node `i`. Update lazily via priority queue; skip nodes already in MST.

Intuition: grow one tree, always swallowing the cheapest edge that reaches a node not yet in it.

```

class Solution {
    public int minCostConnectPoints(int[][] points) {
        int n = points.length;
        boolean[] inMST = new boolean[n];
        int[] minEdge = new int[n];
        Arrays.fill(minEdge, Integer.MAX_VALUE);
        minEdge[0] = 0;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
        pq.offer(new int[]{0, 0});
        int total = 0;
        while (!pq.isEmpty()) {
            int[] current = pq.poll();
            int node = current[0], cost = current[1];
            if (inMST[node]) continue;
            inMST[node] = true;
            total += cost;
            for (int next = 0; next < n; next++) {
                if (!inMST[next]) {
                    int dist = Math.abs(points[node][0]-points[next][0])
                        + Math.abs(points[node][1]-points[next][1]);
                    if (dist < minEdge[next]) {
                        minEdge[next] = dist;
                        pq.offer(new int[]{next, dist});
                    }
                }
            }
        }
        return total;
    }
}

```

Time $O(n^2\log n)$ | **Space** $O(n^2)$

Algorithm Chooser

Signal in problem	Algorithm
Shortest path, unweighted / unit weight	BFS
Shortest path, non-negative weights	Dijkstra
Shortest path, k-hop limit	Bellman-Ford (k+1 rounds)
Shortest path, negative weights	Bellman-Ford (n-1 rounds)
Connected components, cycle detection	Union Find
Directed cycle / valid ordering	Topological Sort (Kahn's)
2-colorable graph	Bipartite BFS coloring
Connect all nodes minimum cost	Prim's / Kruskal's MST
"nearest X for all cells"	Multi-source BFS

Stale Entry Pattern (Dijkstra)

```

int[] current = pq.poll();
int node = current[0], d = current[1];
if (d > dist[node]) continue;    // ← this node was already relaxed to a better distance

```

Without this check, stale entries in the heap cause re-processing and wrong results.
It replaces a `visited[]` array — simpler and correct because a better distance always wins.

Part 7 — Modified Dijkstra / Topological Sort on Chars / Union Find Variants

#1631 Path With Minimum Effort

Description: Find a path from top-left $[0,0]$ to bottom-right $[m-1,n-1]$ that minimizes the maximum absolute difference between adjacent cells.

Variation: Modified Dijkstra. Instead of summing edge weights, the "cost" of a path is the maximum edge cost. Priority queue ordered by effort; $\text{effort}[r][c]$ = minimum over all paths of their max-abs-diff.

```
class Solution {
    public int minimumEffortPath(int[][] heights) {
        int m = heights.length, n = heights[0].length;
        int[][] effort = new int[m][n];
        for (int[] row : effort) Arrays.fill(row, Integer.MAX_VALUE);
        effort[0][0] = 0;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
        pq.offer(new int[]{0, 0, 0}); // [effort, row, col]
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        while (!pq.isEmpty()) {
            int[] current = pq.poll();
            int e = current[0], r = current[1], c = current[2];
            if (r == m-1 && c == n-1) return e;
            if (e > effort[r][c]) continue; // stale entry
            for (int dir = 0; dir < 4; dir++) {
                int nr = r + dr[dir], nc = c + dc[dir];
                if (nr >= 0 && nr < m && nc >= 0 && nc < n) {
                    int newEffort = Math.max(e, Math.abs(heights[nr][nc] - heights[r][c])); // ← VARIATION: max not sum
                    if (newEffort < effort[nr][nc]) {
                        effort[nr][nc] = newEffort;
                        pq.offer(new int[]{newEffort, nr, nc});
                    }
                }
            }
        }
        return 0;
    }
}
```

Time $O(m \cdot n \cdot \log(m \cdot n))$ | **Space** $O(m \cdot n)$

#269 Alien Dictionary

Description: Given a sorted list of alien words, determine the order of characters in the alien alphabet. Return the order as a string, or empty string if invalid.

Variation: Build a directed graph of character ordering. Compare each adjacent pair of words to find the first differing character $\rightarrow$ that gives an edge. Then topological sort. Return empty if there's a cycle or if shorter word is a prefix of longer word in wrong order.

```

class Solution {
    public String alienOrder(String[] words) {
        Map<Character, Set<Character>> graph = new HashMap<>();
        Map<Character, Integer> inDegree = new HashMap<>();
        for (String word : words)
            for (char c : word.toCharArray()) {
                graph.putIfAbsent(c, new HashSet<>());
                inDegree.putIfAbsent(c, 0);
            }
        for (int i = 0; i < words.length - 1; i++) {
            String w1 = words[i], w2 = words[i+1];
            if (w1.length() > w2.length() && w1.startsWith(w2)) return ""; // ← VARIATION: invalid prefix case
            for (int j = 0; j < Math.min(w1.length(), w2.length()); j++) {
                if (w1.charAt(j) != w2.charAt(j)) {
                    if (!graph.get(w1.charAt(j)).contains(w2.charAt(j))) {
                        graph.get(w1.charAt(j)).add(w2.charAt(j));
                        inDegree.merge(w2.charAt(j), 1, Integer::sum);
                    }
                    break;
                }
            }
        }
        Queue<Character> queue = new ArrayDeque<>();
        for (char c : inDegree.keySet()) if (inDegree.get(c) == 0) queue.offer(c);
        StringBuilder result = new StringBuilder();
        while (!queue.isEmpty()) {
            char c = queue.poll();
            result.append(c);
            for (char next : graph.get(c)) {
                inDegree.merge(next, -1, Integer::sum);
                if (inDegree.get(next) == 0) queue.offer(next);
            }
        }
        return result.length() == inDegree.size() ? result.toString() : ""; // ← cycle check
    }
}

```

Time $O(C)$ where C = total characters across all words | **Space** $O(1)$ (at most 26 nodes)

#261 Graph Valid Tree

Description: Given n nodes labeled $0..n-1$ and a list of undirected edges, determine if they form a valid tree (connected, no cycles).

Variation: A valid tree has exactly $n-1$ edges. Use Union Find to detect cycles. If any edge connects two nodes already in the same component → cycle → not a tree.

```

class Solution {
    int[] parent, rank;

    public boolean validTree(int n, int[][] edges) {
        if (edges.length != n - 1) return false;    // ← VARIATION: tree must have exactly n-1 edges
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
        for (int[] edge : edges)
            if (!union(edge[0], edge[1])) return false; // ← cycle detected → not a tree
        return true;
    }

    int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }

    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }
}

```

Time $O(n \cdot \alpha(n)) \approx O(n)$ | **Space** $O(n)$

#323 Number of Connected Components in an Undirected Graph

Description: Given n nodes and a list of undirected edges, return the number of connected components.

Algorithm: Union Find. Start with n components. Each successful union (merging two previously separate components) decrements the count by 1.

```

class Solution {
    int[] parent, rank;

    public int countComponents(int n, int[][] edges) {
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
        int components = n;                // ← start with n isolated components
        for (int[] edge : edges)
            if (union(edge[0], edge[1])) components--;    // ← VARIATION: decrement on merge
        return components;
    }

    int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }

    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
        parent[py] = px;
        if (rank[px] == rank[py]) rank[px]++;
        return true;
    }
}

```

Time $O(n \cdot \alpha(n)) \approx O(n)$ | **Space** $O(n)$

#286 Walls and Gates

Description: Each cell is a gate (0), wall (-1), or empty (INF = 2147483647). Fill each empty room with distance to its nearest gate. **Variation:** multi-source BFS — seed the queue with every gate at distance 0, then expand outward simultaneously.

```

class Solution {
    public void wallsAndGates(int[][] rooms) {
        if (rooms.length == 0) return;
        int m = rooms.length, n = rooms[0].length;
        Queue<int[]> queue = new ArrayDeque<>();
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                if (rooms[i][j] == 0) queue.offer(new int[]{i, j}); // ← VARIATION: seed ALL gates
        int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
        while (!queue.isEmpty()) {
            int[] cell = queue.poll();
            for (int dir = 0; dir < 4; dir++) {
                int ni = cell[0] + dr[dir], nj = cell[1] + dc[dir];
                if (ni < 0 || ni >= m || nj < 0 || nj >= n || rooms[ni][nj] != Integer.MAX_VALUE) continue;
                rooms[ni][nj] = rooms[cell[0]][cell[1]] + 1;
                queue.offer(new int[]{ni, nj});
            }
        }
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#721 Accounts Merge

Description: Each account has a name and a list of emails. Merge accounts that share any email. Return merged accounts with emails sorted.

Variation: Union Find over emails. Assign each unique email an integer id, union all emails within the same account, then group emails by their root.

```

class Solution {
    private int[] parent;
    public List<List<String>> accountsMerge(List<List<String>> accounts) {
        Map<String, Integer> emailToId = new HashMap<>();
        Map<String, String> emailToName = new HashMap<>();
        int id = 0;
        for (List<String> account : accounts)
            for (int i = 1; i < account.size(); i++) {
                String email = account.get(i);
                if (!emailToId.containsKey(email)) emailToId.put(email, id++);
                emailToName.put(email, account.get(0));
            }
        parent = new int[id];
        for (int i = 0; i < id; i++) parent[i] = i;
        for (List<String> account : accounts)
            for (int i = 2; i < account.size(); i++)
                union(emailToId.get(account.get(1)), emailToId.get(account.get(i))); // ← VARIATION: union emails in account
        Map<Integer, List<String>> groups = new HashMap<>();
        for (String email : emailToId.keySet())
            groups.computeIfAbsent(find(emailToId.get(email)), x -> new ArrayList<>()).add(email);
        List<List<String>> result = new ArrayList<>();
        for (List<String> emails : groups.values()) {
            Collections.sort(emails);
            List<String> merged = new ArrayList<>();
            merged.add(emailToName.get(emails.get(0)));
            merged.addAll(emails);
            result.add(merged);
        }
        return result;
    }
    private int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }
    private void union(int x, int y) { parent[find(x)] = find(y); }
}

```

Time $O(n \cdot \alpha)$ with sorting $O(n \log n)$ | **Space** $O(n)$

#827 Making A Large Island

Description: In a binary grid you may change at most one 0 to 1. Return the size of the largest island possible. **Variation:** two passes. First DFS-color each island with a unique id (starting at 2) and record its size. Then for every 0, sum the sizes of distinct neighboring islands + 1.

```
class Solution {
    private int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};
    public int largestIsland(int[][] grid) {
        int n = grid.length, id = 2;
        Map<Integer, Integer> size = new HashMap<>();
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (grid[i][j] == 1) size.put(id, dfs(grid, i, j, id++)); // ← VARIATION: color + record size
        int max = size.values().stream().mapToInt(x -> x).max().orElse(0);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (grid[i][j] == 0) {
                    Set<Integer> seen = new HashSet<>();
                    int total = 1;
                    for (int dir = 0; dir < 4; dir++) {
                        int ni = i + dr[dir], nj = j + dc[dir];
                        if (ni < 0 || ni >= n || nj < 0 || nj >= n || grid[ni][nj] <= 1) continue;
                        if (seen.add(grid[ni][nj])) total += size.get(grid[ni][nj]); // ← VARIATION: sum distinct neighbors
                    }
                    max = Math.max(max, total);
                }
        return max;
    }
    private int dfs(int[][] grid, int i, int j, int id) {
        if (i < 0 || i >= grid.length || j < 0 || j >= grid.length || grid[i][j] != 1) return 0;
        grid[i][j] = id;
        int count = 1;
        for (int dir = 0; dir < 4; dir++) count += dfs(grid, i + dr[dir], j + dc[dir], id);
        return count;
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#909 Snakes and Ladders

Description: On an $n \times n$ board numbered in boustrophedon (zigzag) order, find the minimum dice moves (1-6) from square 1 to square n^2 . Ladders/snakes teleport you. **Variation:** BFS over square numbers. Convert a square number to (row, col) accounting for the zigzag layout.

```

class Solution {
    public int snakesAndLadders(int[][] board) {
        int n = board.length;
        boolean[] visited = new boolean[n * n + 1];
        Queue<Integer> queue = new ArrayDeque<>();
        queue.offer(1);
        visited[1] = true;
        int moves = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int s = 0; s < size; s++) {
                int current = queue.poll();
                if (current == n * n) return moves;
                for (int k = 1; k <= 6 && current + k <= n * n; k++) {
                    int next = current + k;
                    int[] rc = toCoord(next, n);          // ← VARIATION: number → zigzag coordinate
                    if (board[rc[0]][rc[1]] != -1) next = board[rc[0]][rc[1]]; // snake/ladder
                    if (!visited[next]) { visited[next] = true; queue.offer(next); }
                }
            }
            moves++;
        }
        return -1;
    }
    private int[] toCoord(int num, int n) {
        int row = (num - 1) / n, col = (num - 1) % n;
        if (row % 2 == 1) col = n - 1 - col;           // ← VARIATION: reverse on odd rows
        return new int[]{n - 1 - row, col};
    }
}

```

Time $O(n^2)$ | **Space** $O(n^2)$

#1091 Shortest Path in Binary Matrix

Description: In an $n \times n$ binary grid, find the length of the shortest clear path (cells of 0) from top-left to bottom-right, moving in 8 directions. **Variation:** standard BFS but with 8-directional movement (includes diagonals).

```

class Solution {
    public int shortestPathBinaryMatrix(int[][] grid) {
        int n = grid.length;
        if (grid[0][0] == 1 || grid[n-1][n-1] == 1) return -1;
        int[] dr = {1,-1,0,0,1,1,-1,-1}, dc = {0,0,1,-1,1,1,-1,-1}; // ← VARIATION: 8 directions
        Queue<int[]> queue = new ArrayDeque<>();
        queue.offer(new int[]{0, 0});
        grid[0][0] = 1;
        int path = 1;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int s = 0; s < size; s++) {
                int[] cell = queue.poll();
                if (cell[0] == n-1 && cell[1] == n-1) return path;
                for (int dir = 0; dir < 8; dir++) {
                    int ni = cell[0] + dr[dir], nj = cell[1] + dc[dir];
                    if (ni < 0 || ni >= n || nj < 0 || nj >= n || grid[ni][nj] != 0) continue;
                    grid[ni][nj] = 1;
                    queue.offer(new int[]{ni, nj});
                }
            }
            path++;
        }
        return -1;
    }
}

```


Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$